

Spring Core & Maven

Exercise 1: Configuring a Basic Spring Application

pom.xml :

```
<dependencies>

    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
        <version>5.3.30</version>
    </dependency>

</dependencies>
```

BookRepository.java :

```
package com.library.repository;

public class BookRepository {

    public void display() {
        System.out.println("Book Repository Created");
    }
}
```

BookService.java :

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void showService() {
        System.out.println("Book Service Created");
        bookRepository.display();
    }
}
```

applicationContext.xml :

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="
            http://www.springframework.org/schema/beans
            https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository"
        class="com.library.repository.BookRepository"/>
```

```

<bean id="bookService"
      class="com.library.service.BookService">

    <property name="bookRepository"
              ref="bookRepository"/>

</bean>

</beans>

```

MainApp.java :

```

package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import com.library.service.BookService;

public class MainApp {

    public static void main(String[] args) {

        ApplicationContext context =
            new ClassPathXmlApplicationContext(
                "applicationContext.xml");

        BookService service =
            (BookService) context.getBean("bookService");

        service.showService();
    }
}

```

Output :

```

Book Service Created
Book Repository Created

```

Exercise 2: Implementing Dependency Injection

BookRepository.java :

```

package com.library.repository;

public class BookRepository {

    public void display() {
        System.out.println("Book Repository Connected");
    }
}

```

BookService.java :

```

package com.library.service;

import com.library.repository.BookRepository;

```

```

public class BookService {

    private BookRepository bookRepository;

    // Setter Injection
    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void showService() {
        System.out.println("Book Service Running");
        bookRepository.display();
    }
}

```

applicationContext.xml :

```

<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="
            http://www.springframework.org/schema/beans
            https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository"
        class="com.library.repository.BookRepository"/>

    <bean id="bookService"
        class="com.library.service.BookService">

        <property name="bookRepository"
            ref="bookRepository"/>

    </bean>

</beans>

```

LibraryManagementApplication.java :

```

package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import com.library.service.BookService;

public class LibraryManagementApplication {

    public static void main(String[] args) {

        ApplicationContext context =
            new ClassPathXmlApplicationContext(
                "applicationContext.xml");

        BookService service =
            (BookService) context.getBean("bookService");

        service.showService();
    }
}

```

```
}  
}
```

Output :

Book Service Running
Book Repository Connected

Exercise 3: Implementing Logging with Spring AOP

pom.xml :

```
<dependency>  
  <groupId>org.springframework</groupId>  
  <artifactId>spring-aop</artifactId>  
  <version>5.3.30</version>  
</dependency>  
  
<dependency>  
  <groupId>org.aspectj</groupId>  
  <artifactId>aspectjweaver</artifactId>  
  <version>1.9.21</version>  
</dependency>
```

BookService.java :

```
package com.library.service;  
  
public class BookService {  
  
    public void showService() {  
        System.out.println("Book Service Running");  
    }  
}
```

LoggingAspect.java :

```
package com.library.aspect;  
  
import org.aspectj.lang.ProceedingJoinPoint;  
  
public class LoggingAspect {  
  
    public Object logExecutionTime(  
        ProceedingJoinPoint joinPoint) throws Throwable {  
  
        long start = System.currentTimeMillis();  
  
        Object result = joinPoint.proceed();  
  
        long end = System.currentTimeMillis();  
  
        System.out.println(  
            joinPoint.getSignature().getName()  
            + " executed in "  
            + (end - start) + " ms");  
  
        return result;  
    }  
}
```

```
}  
}
```

applicationContext.xml :

```
<?xml version="1.0" encoding="UTF-8"?>  
  
<beans xmlns="http://www.springframework.org/schema/beans"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xmlns:aop="http://www.springframework.org/schema/aop"  
    xsi:schemaLocation="  
        http://www.springframework.org/schema/beans  
        https://www.springframework.org/schema/beans/spring-beans.xsd  
        http://www.springframework.org/schema/aop  
        https://www.springframework.org/schema/aop/spring-aop.xsd">  
  
    <bean id="bookService"  
        class="com.library.service.BookService"/>  
  
    <bean id="loggingAspect"  
        class="com.library.aspect.LoggingAspect"/>  
  
    <aop:config>  
  
        <aop:aspect ref="loggingAspect">  
  
            <aop:around  
                method="logExecutionTime"  
                pointcut="execution(* com.library.service.*(..))"/>  
  
            </aop:aspect>  
  
        </aop:config>  
  
    </beans>
```

LibraryManagementApplication.java :

```
package com.library;  
  
import org.springframework.context.ApplicationContext;  
import org.springframework.context.support.ClassPathXmlApplicationContext;  
  
import com.library.service.BookService;  
  
public class LibraryManagementApplication {  
  
    public static void main(String[] args) {  
  
        ApplicationContext context =  
            new ClassPathXmlApplicationContext(  
                "applicationContext.xml");  
  
        BookService service =  
            (BookService) context.getBean("bookService");  
  
        service.showService();  
    }  
}
```

Output :

Book Service Running
showService executed in 1 ms

Exercise 4: Creating and Configuring a Maven Project

pom.xml :

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.library</groupId>
  <artifactId>LibraryManagement</artifactId>
  <version>1.0</version>

  <dependencies>

    <!-- Spring Context -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>5.3.30</version>
    </dependency>

    <!-- Spring AOP -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-aop</artifactId>
      <version>5.3.30</version>
    </dependency>

    <!-- Spring Web MVC -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-webmvc</artifactId>
      <version>5.3.30</version>
    </dependency>

  </dependencies>

  <build>
    <plugins>

      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.8.1</version>

        <configuration>
          <source>1.8</source>
          <target>1.8</target>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

```
        </plugin>
    </plugins>
</build>

</project>
```

Project Structure :

```
LibraryManagement
├── src/main/java
├── src/main/resources
└── pom.xml
```

Maven Commands USED :

```
mvn clean
mvn compile
mvn package
```

Output :

BUILD SUCCES

Exercise 5: Configuring the Spring IoC Container

BookRepository.java :

```
package com.library.repository;

public class BookRepository {

    public void display() {
        System.out.println("Book Repository Created");
    }
}
```

BookService.java :

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void showService() {
        System.out.println("Book Service Created");
        bookRepository.display();
    }
}
```

```
}  
}
```

applicationContext.xml :

```
<?xml version="1.0" encoding="UTF-8"?>  
  
<beans xmlns="http://www.springframework.org/schema/beans"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="  
        http://www.springframework.org/schema/beans  
        https://www.springframework.org/schema/beans/spring-beans.xsd">  
  
    <bean id="bookRepository"  
        class="com.library.repository.BookRepository"/>  
  
    <bean id="bookService"  
        class="com.library.service.BookService">  
  
        <property name="bookRepository"  
            ref="bookRepository"/>  
  
    </bean>  
  
</beans>
```

LibraryManagementApplication.java :

```
package com.library;  
  
import org.springframework.context.ApplicationContext;  
import org.springframework.context.support.ClassPathXmlApplicationContext;  
  
import com.library.service.BookService;  
  
public class LibraryManagementApplication {  
  
    public static void main(String[] args) {  
  
        ApplicationContext context =  
            new ClassPathXmlApplicationContext(  
                "applicationContext.xml");  
  
        BookService service =  
            (BookService) context.getBean("bookService");  
  
        service.showService();  
    }  
}
```

Output :

```
Book Service Created  
Book Repository Created
```

Exercise 6: Configuring Beans with Annotations

BookRepository.java :


```

package com.library.repository;

import org.springframework.stereotype.Repository;

@Repository
public class BookRepository {

    public void display() {
        System.out.println("Book Repository Created");
    }
}

```

BookService.java :

```

package com.library.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.library.repository.BookRepository;

@Service
public class BookService {

    @Autowired
    private BookRepository bookRepository;

    public void showService() {
        System.out.println("Book Service Created");
        bookRepository.display();
    }
}

```

applicationContext.xml :

```

<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        https://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context
        https://www.springframework.org/schema/context/spring-context.xsd">

    <context:component-scan base-package="com.library"/>

</beans>

```

LibraryManagementApplication.java :

```

package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import com.library.service.BookService;

```

```

public class LibraryManagementApplication {

    public static void main(String[] args) {

        ApplicationContext context =
            new ClassPathXmlApplicationContext(
                "applicationContext.xml");

        BookService service =
            context.getBean(BookService.class);

        service.showService();
    }
}

```

Output :

```

Book Service Created
Book Repository Created

```

Exercise 7: Implementing Constructor and Setter Injection

BookRepository.java :

```

package com.library.repository;

public class BookRepository {

    public void display() {
        System.out.println("Book Repository Created");
    }
}

```

BookService.java :

```

package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    // Constructor Injection
    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
        System.out.println("Constructor Injection Done");
    }

    // Setter Injection
    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
        System.out.println("Setter Injection Done");
    }

    public void showService() {
        System.out.println("Book Service Created");
        bookRepository.display();
    }
}

```

```
}  
}
```

applicationContext.xml :

```
<?xml version="1.0" encoding="UTF-8"?>  
  
<beans xmlns="http://www.springframework.org/schema/beans"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="  
        http://www.springframework.org/schema/beans  
        https://www.springframework.org/schema/beans/spring-beans.xsd">  
  
    <bean id="bookRepository"  
        class="com.library.repository.BookRepository"/>  
  
    <bean id="bookService"  
        class="com.library.service.BookService">  
  
        <!-- Constructor Injection -->  
        <constructor-arg ref="bookRepository"/>  
  
        <!-- Setter Injection -->  
        <property name="bookRepository"  
            ref="bookRepository"/>  
  
    </bean>  
  
</beans>
```

LibraryManagementApplication.java :

```
package com.library;  
  
import org.springframework.context.ApplicationContext;  
import org.springframework.context.support.ClassPathXmlApplicationContext;  
  
import com.library.service.BookService;  
  
public class LibraryManagementApplication {  
  
    public static void main(String[] args) {  
  
        ApplicationContext context =  
            new ClassPathXmlApplicationContext(  
                "applicationContext.xml");  
  
        BookService service =  
            (BookService) context.getBean("bookService");  
  
        service.showService();  
    }  
}
```

Output :

```
Constructor Injection Done  
Setter Injection Done  
Book Service Created
```

Exercise 8: Implementing Basic AOP with Spring

pom.xml :

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>5.3.30</version>
</dependency>

<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-aop</artifactId>
  <version>5.3.30</version>
</dependency>

<dependency>
  <groupId>org.aspectj</groupId>
  <artifactId>aspectjweaver</artifactId>
  <version>1.9.21</version>
</dependency>
```

BookService.java :

```
package com.library.service;

public class BookService {

    public void showService() {
        System.out.println("Book Service Running");
    }
}
```

LoggingAspect.java :

```
package com.library.aspect;

public class LoggingAspect {

    public void beforeAdvice() {
        System.out.println("Before method execution");
    }

    public void afterAdvice() {
        System.out.println("After method execution");
    }
}
```

applicationContext.xml :

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:aop="http://www.springframework.org/schema/aop"
  xsi:schemaLocation="
```

```
http://www.springframework.org/schema/beans
https://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/aop
https://www.springframework.org/schema/aop/spring-aop.xsd">
```

```
<bean id="bookService"
      class="com.library.service.BookService"/>

<bean id="loggingAspect"
      class="com.library.aspect.LoggingAspect"/>

<aop:config>

  <aop:aspect ref="loggingAspect">

    <aop:before
      method="beforeAdvice"
      pointcut="execution(* com.library.service.*(..))"/>

    <aop:after
      method="afterAdvice"
      pointcut="execution(* com.library.service.*(..))"/>

  </aop:aspect>

</aop:config>

</beans>
```

LibraryManagementApplication.java :

```
package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import com.library.service.BookService;

public class LibraryManagementApplication {

    public static void main(String[] args) {

        ApplicationContext context =
            new ClassPathXmlApplicationContext(
                "applicationContext.xml");

        BookService service =
            (BookService) context.getBean("bookService");

        service.showService();
    }
}
```

Output :

```
Before method execution
Book Service Running
After method execution
```

Exercise 9: Creating a Spring Boot Application

pom.xml :

```
<dependencies>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>

</dependencies>
```

application.properties :

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver

spring.jpa.hibernate.ddl-auto=update

spring.h2.console.enabled=true
```

Book.java :

```
package com.library.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;

@Entity
public class Book {

    @Id
    private Long id;
    private String title;

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }
}
```

```

    public void setTitle(String title) {
        this.title = title;
    }
}

```

BookRepository.java :

```

package com.library.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.library.entity.Book;

public interface BookRepository
    extends JpaRepository<Book, Long> {
}

```

BookController.java :

```

package com.library.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.library.entity.Book;
import com.library.repository.BookRepository;

@RestController
@RequestMapping("/books")
public class BookController {

    @Autowired
    private BookRepository repository;

    @GetMapping
    public List<Book> getAllBooks() {
        return repository.findAll();
    }

    @PostMapping
    public Book addBook(@RequestBody Book book) {
        return repository.save(book);
    }
}

```

LibraryManagementApplication.java :

```

package com.library;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class LibraryManagementApplication {

    public static void main(String[] args) {

        SpringApplication.run(
            LibraryManagementApplication.class,

```

```
        args);  
    }  
}
```

Test REST Endpoint :

POST :

POST <http://localhost:8080/books>

```
{  
  "id": 1,  
  "title": "Spring Boot"  
}
```

GET :

GET <http://localhost:8080/books>

Output :

```
[  
  {  
    "id": 1,  
    "title": "Spring Boot"  
  }  
]
```